Universidad Autónoma Gabriel René Moreno



CASO DE ESTUDIO :Clasificador del sentimiento de reseñas de productos, hoteles o restaurantes

INTEGRANTES:

URGEL YOSHIIE KENY LEONARDO 219055254

NICOLAS JUNIOR VERDUGUEZ DURAN 223044784

ARZE KAO LUIS ANGEL 222007508

ANGELES ROSALES COLQUE 223044172

Índice

1. Introducción 3
1.1. Definición del problema 3
1.2. Objetivos 3
2. Organización del equipo y metodología 3
2.1. Metodología de trabajo 4
2.2. Arquitectura general 4
3. Integrante 1 – Datos y NLP 5
3.1. Conjunto de datos 5
3.2. Análisis exploratorio (EDA) 5
3.3. Preprocesamiento del texto 7
3.4. Stemming vs. lematización 7
4. Integrante 2 – Modelado y pipeline 8
4.1. División de los datos 8
4.2. Representación numérica con TF-IDF 8
4.3. Experimentos comparativos 9
4.4. Pipeline y optimización con GridSearchCV 9
4.5. Evaluación final 10
4.6. Interpretación del modelo 11
4.7. Análisis de errores 11
4.8. Exportación de artefactos 12
5. Integrante 3 – Backend (API REST con FastAPI) 12
5.1. Objetivo 12
5.2. Estructura y conexión con el equipo 12
5.3. Rutas de la API 13
5.4. Paso a paso del NLP (Requisito 5.2) 13
5.5. Validaciones y manejo de errores 14
5.6. CORS 14
5.7. Funciones adicionales y pruebas 14
6. Integrante 4 – Frontend (interfaz web) 14
6.1. Tecnologías y estructura 14
6.2. Pantallas de la aplicación 15
6.3. Cumplimiento del paso a paso (Requisito 5.2) 19
6.4. Validaciones, accesibilidad y seguridad 19
7. Integración y verificación 20
8. Resultados y discusión 20
9. Limitaciones y trabajo futuro 21
10. Conclusiones 21
11. Declaración de uso de inteligencia artificial 21
12. Contribuciones e historial 22
Referencias 22

1. Introducción

Las plataformas de turismo reciben miles de opiniones escritas por sus usuarios cada día. Leerlas una por una para saber si un hotel deja a sus clientes satisfechos o no es lento y costoso. El Procesamiento del Lenguaje Natural (NLP) y el aprendizaje automático permiten automatizar esa lectura: un modelo puede aprender, a partir de reseñas ya calificadas, qué palabras y expresiones acompañan a una experiencia buena o mala.

Este informe documenta el desarrollo de **Opinómetro**, una aplicación web que clasifica el sentimiento (Positivo o Negativo) de reseñas de hoteles escritas en español y que muestra, paso a paso, cómo se transforma el texto antes de llegar al modelo. El proyecto se construyó en cuatro etapas encadenadas (datos y NLP, modelado, backend y frontend), cada una a cargo de un integrante del equipo.

1.1. Definición del problema

Dada una reseña en texto libre, determinar si la opinión del autor es **positiva** o **negativa** y con qué probabilidad. Se trata de un problema de clasificación binaria supervisada sobre texto. La consigna del caso de estudio pide además que la aplicación sea transparente: la interfaz debe mostrar en pantalla cada etapa de la limpieza del texto (normalización, tokens, eliminación de stop words, stemming o lematización) junto al veredicto final.

1.2. Objetivos

Objetivo general. Desarrollar una aplicación web que clasifique el sentimiento de reseñas en español mediante un pipeline reproducible de NLP y aprendizaje automático, y que exponga el proceso de limpieza de forma visual.

Objetivos específicos:

- Reunir un conjunto de al menos 1000 reseñas reales en español, balanceado entre clases, y explorarlo estadísticamente.
- Implementar la limpieza del texto (minúsculas, tildes, signos, stop words) conservando las negaciones, y comparar stemming con lematización.
- Entrenar y comparar modelos con TF-IDF (unigramas vs. bigramas; Regresión Logística vs. Naive Bayes) y optimizar el mejor con GridSearchCV.
- Servir el modelo mediante una API REST con FastAPI, con validaciones y rutas de predicción, métricas y estado.
- Construir una interfaz web con HTML, CSS y JavaScript que muestre el paso a paso del NLP y pinte el resultado de verde o rojo.

2. Organización del equipo y metodología

El trabajo se dividió en cuatro roles con carpetas y ramas de Git separadas, para que cada integrante pudiera avanzar en paralelo y unir su parte mediante *pull requests*.

Tabla 1. Distribución de roles del equipo.

Integrante	Rol	Carpetas	Rama de Git
Keny Leonardo Urgel Yoshiie (219055254)	1 · Datos y NLP	`data/`, `notebooks/`, `src/nlp/`	`feature/nlp-preprocess`
Nicolás	2 · Modelado y pipeline	`src/models/`	`ramaNico`
Luis Angel Arze Kao (222007508)	3 · Backend (API REST)	`api/`	`feature/backend-api`
Angela	4 · Frontend (interfaz web)	`frontend/`	`anghy`

2.1. Metodología de trabajo

Se siguió un flujo **en cascada**: cada etapa produce un artefacto que consume la siguiente. Para no bloquearse entre sí, el equipo usó dos acuerdos: (1) el backend arrancó con una respuesta simulada fija ("Positivo – 90 %") para que el frontend pudiera conectarse desde el primer día, y (2) el modelado preparó la estructura de `train.py` mientras llegaban las funciones de limpieza. El código se versionó en GitHub; cada integrante trabajó en su rama y la integración se hizo con *merges* revisados.

2.2. Arquitectura general

Una reseña recorre cuatro componentes. Las dos primeras etapas ocurren una sola vez, durante el entrenamiento; las dos últimas, en cada consulta del usuario.

Tabla 2. Flujo de datos entre los componentes.

Etapa	Componente	Entrada	Salida
1. Limpieza (Keny)	`src/nlp/preprocess.py`	Texto crudo	Texto normalizado, sin stop words, con raíces
2. Modelo (Nicolás)	`src/models/train.py`	`data/resenas.csv`	`pipeline_modelo.pkl` y `metricas.json`
3. API (Luis Angel)	`api/main.py`	JSON `{"texto": ...}`	Veredicto, probabilidades y pasos del NLP
4. Interfaz (Angela)	`frontend/`	Reseña escrita por el usuario	Resultado en verde/rojo y recorrido visual

Los integrantes 1 a 3 trabajaron en Python, porque las bibliotecas de NLP y aprendizaje automático vistas en clase (scikit-learn, NLTK, pandas) están en ese lenguaje. El frontend usa JavaScript, ya que es lo que entiende el navegador, y se comunica con Python mediante la función `fetch()`.

Tabla 3. Tecnologías utilizadas.

Capa	Tecnologías
Datos y NLP	Python 3, pandas, NLTK (stop words en español, SnowballStemmer), Jupyter
Modelado	scikit-learn 1.9 (TfidfVectorizer, LogisticRegression, MultinomialNB, Pipeline, GridSearchCV), joblib
Backend	FastAPI, Uvicorn, Pydantic, pytest + httpx
Frontend	HTML5, CSS3, JavaScript (módulos ES y Fetch API), sin frameworks
Colaboración	Git y GitHub (ramas por integrante y pull requests)

3. Integrante 1 – Datos y NLP

Responsable: Keny Leonardo Urgel Yoshiie (219055254) · **Carpetas:** `data/`, `notebooks/`, `src/nlp/`

3.1. Conjunto de datos

Se usan reseñas reales del corpus **Rest-Mex 2025**, publicado en Hugging Face con licencia CC BY 4.0, que reúne opiniones de TripAdvisor sobre destinos turísticos de México. Cada reseña trae un título, el texto y una calificación de 1 a 5 estrellas. El script `descargar_datos.py` prepara el conjunto final así:

- Se conservan solo las reseñas de tipo **Hotel**.
- Las estrellas se convierten en sentimiento binario: 1–2 estrellas = **Negativo (0)**, 4–5 estrellas = **Positivo (1)**. Las de 3 estrellas se descartan por ser neutras.
- El texto final une título y reseña. Se reparan problemas de codificación (filas mezcladas UTF-8/Latin-1) y se eliminan duplicados.
- Se toma una muestra balanceada y reproducible de **500 reseñas por clase** (semilla 42), para un total de **1000 reseñas**.

En una primera versión el equipo trabajó con un conjunto sintético. Su exactitud resultaba artificialmente alta por falta de variedad, y se intentó corregirlo agregando ruido. Finalmente se reemplazó por estas reseñas reales, que reflejan mejor el lenguaje de los usuarios.

3.2. Análisis exploratorio (EDA)

El conjunto no tiene valores nulos ni textos duplicados, y las dos clases están perfectamente balanceadas (Figura 1). Como hay la misma cantidad de ejemplos de cada clase, la exactitud es una métrica justa: un modelo que respondiera siempre lo mismo acertaría solo el 50 %.

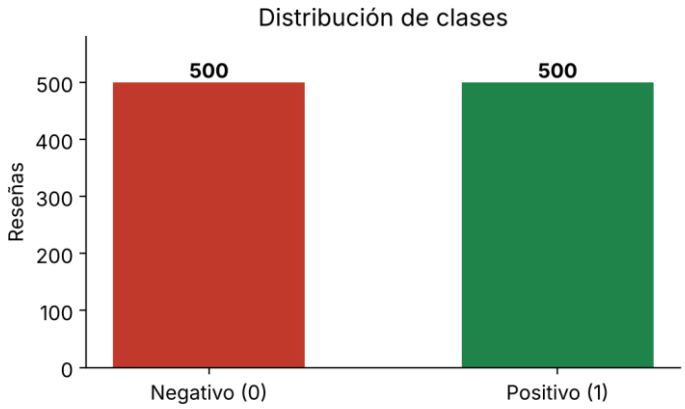


Figura 1. Distribución de reseñas por sentimiento.

Las reseñas negativas son, en promedio, más largas: **108 palabras** frente a **82** de las positivas (Figura 2). Quien tuvo una mala experiencia tiende a explicar con detalle qué falló, mientras que una buena estancia suele resumirse en pocas frases.

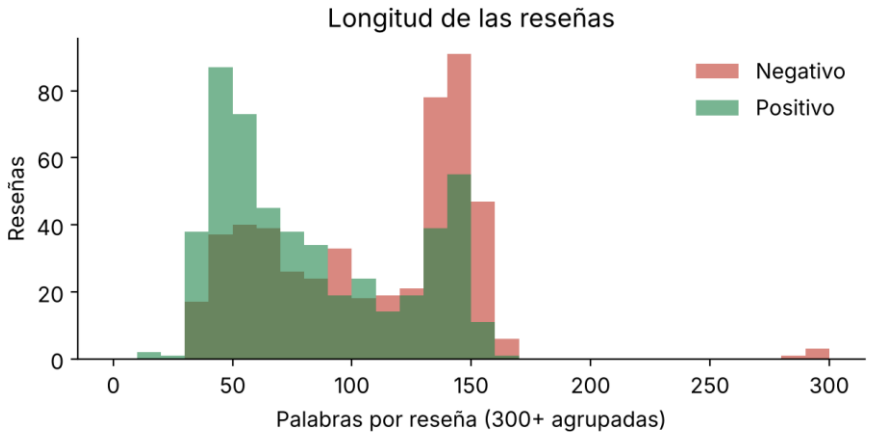


Figura 2. Distribución de la longitud de las reseñas por clase.

La Figura 3 muestra las palabras más frecuentes de cada clase después de la limpieza. Palabras como *hotel*, *habitación* o *servicio* aparecen en ambas y aportan poco para distinguirlas. En cambio, *excelente*, *mejor*, *bien*, *buena* y *amable* dominan en las positivas, y *mal*, *nada*, *agua* o *recepción* en las negativas. La palabra *pero* es muy común en ambas clases: anticipa matices y explica parte de los errores del modelo (sección 4.7).

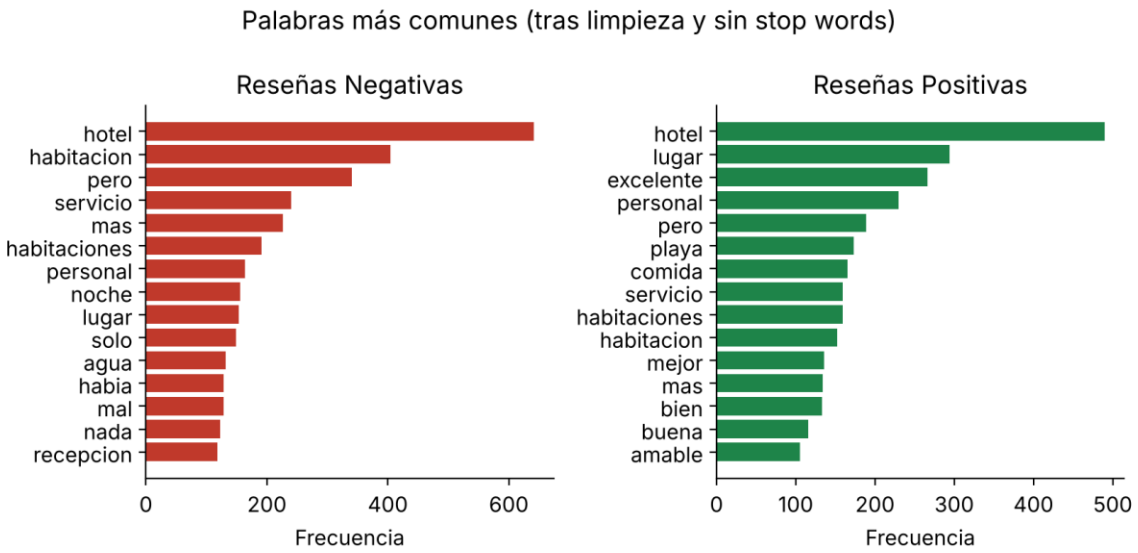


Figura 3. Las 15 palabras más frecuentes en cada clase (sin stop words).

La negación también es informativa: la palabra **"no"** aparece en 448 de las 500 reseñas negativas, pero solo en 210 de las positivas. Por eso la limpieza no debe eliminarla.

3.3. Preprocesamiento del texto

La clase `NLPPreprocessor` (`src/nlp/preprocess.py`) concentra toda la limpieza. La misma clase se usa al entrenar, dentro del pipeline del modelo y en la API, así que el texto siempre se procesa igual.

Tabla 4. Etapas de la limpieza implementadas en NLPPreprocessor.

Paso	Operación	Ejemplo
1	Pasar a minúsculas	"PÉSIMO" → "pésimo"
2	Quitar URLs (`http…`, `www…`)	"ver www.hotel.com" → "ver "
3	Quitar tildes con normalización Unicode NFD	"pésimo" → "pesimo"
4	Quitar signos, números y emojis (solo quedan letras a–z)	"¡¡pesimo!!" → "pesimo"
5	Separar en tokens por espacios	"no lo recomiendo" → ["no", "lo", "recomiendo"]
6	Eliminar stop words de NLTK, salvo negaciones	["no", "lo", "recomiendo"] → ["no", "recomiendo"]
7	Reducir cada palabra a su raíz (stemming) o a su lema	"recomiendo" → "recom"

La lista de stop words en español de NLTK incluye palabras como *no*, *ni* o *nada*. Si se eliminaran, "no es bueno" y "es bueno" quedarían iguales. Por eso se retiran de la lista las negaciones y contrastes **no, ni, tampoco, nunca, jamás, nada** y **pero**. Con eso, la lista final tiene 309 stop words. Tras la limpieza, el corpus pasa de 95 201 a 50 194 palabras: casi la mitad del texto eran conectores sin carga de sentimiento.

3.4. Stemming vs. lematización

La consigna pide evaluar dos formas de agrupar las variantes de una palabra:

- **Stemming** (SnowballStemmer de NLTK): corta sufijos con reglas fijas. Es rápido, pero la raíz puede no ser una palabra real ("excelente" → "excelent", "limpia" → "limpi").
- **Lematización**: devuelve la forma de diccionario de la palabra. Se implementó con un diccionario propio de lemas comunes de opinión ("buenísimo", "buenos" → "bueno"; "pésima" → "pésimo"); las palabras que no están en él quedan igual.

Tabla 5. Comparación de stemming y lematización con la misma configuración del clasificador (C = 10, max_df = 0.9, unigramas).

Técnica	Vocabulario del corpus	Vocabulario TF-IDF (entrenamiento)	F1 en validación cruzada (5-fold)	Accuracy en prueba
Sin reducción (solo limpieza)	8 325	–	–	–
Lematización (diccionario)	8 314	7 347	0.917	0.935
Stemming (Snowball)	**4 846**	**4 357**	**0.926**	**0.935**

El stemming reduce el vocabulario en un **42 %** porque agrupa formas como *limpia*, *limpias*, *limpio* y *limpieza* en la raíz "limpi", lo que concentra la evidencia en menos columnas. La lematización con diccionario apenas cambia el vocabulario, porque cubre pocas palabras. Las dos técnicas empatan en el conjunto de prueba, pero el stemming obtiene mejor F1 en validación cruzada, que es una estimación más estable. Por eso GridSearchCV lo eligió para el modelo final (sección 4.4).

4. Integrante 2 – Modelado y pipeline

Responsable: Nicolás · **Carpeta:** `src/models/` · **Script:** `train.py`

4.1. División de los datos

Las 1000 reseñas se dividen en **800 de entrenamiento** y **200 de prueba** (80/20), con `train_test_split` estratificado y semilla 42. La estratificación mantiene el 50/50 de clases en ambos grupos. El conjunto de prueba no se usa en ningún momento del entrenamiento ni de la búsqueda de hiperparámetros: solo sirve para la evaluación final.

4.2. Representación numérica con TF-IDF

Un modelo no puede leer texto, así que `TfidfVectorizer` convierte cada reseña en un vector numérico. Cada columna corresponde a una palabra (o raíz) del vocabulario, y su valor es:

```
TF-IDF(t, d) = TF(t, d) × IDF(t)
IDF(t) = ln( (1 + N) / (1 + df(t)) ) + 1
```

TF es cuántas veces aparece el término *t* en la reseña *d*; *N* es el número de reseñas y *df(t)* en cuántas aparece *t*. Así, una palabra repetida en una reseña pesa más, pero una palabra que aparece en casi todas (como "hotel") pesa menos, porque no ayuda a distinguir. Cada vector se normaliza (norma L2) para que las reseñas largas no dominen sobre las cortas.

Con **unigramas** cada columna es una palabra suelta. Con **bigramas** se agregan también pares de palabras consecutivas, como "no recomiend" o "muy limpi", que capturan algo de contexto a cambio de casi duplicar el vocabulario.

4.3. Experimentos comparativos

Para responder qué combinación funciona mejor, se entrenaron cuatro modelos sobre el texto lematizado y se evaluaron en el conjunto de prueba:

- **Regresión Logística**: aprende un peso por palabra y combina los pesos con la función sigmoide para obtener la probabilidad de la clase positiva.
- **Naive Bayes multinomial**: calcula la probabilidad de cada palabra dentro de cada clase y las multiplica, suponiendo que las palabras son independientes entre sí.

Tabla 6. Resultados de los experimentos (lematización, min_df = 2, conjunto de prueba).

Modelo	N-gramas	Vocabulario	Accuracy	F1 ponderado
Regresión Logística	Unigramas	3 242	0.925	0.925
Regresión Logística	Uni + Bigramas	6 247	0.935	0.935
Naive Bayes	Unigramas	3 242	0.930	0.930
Naive Bayes	Uni + Bigramas	6 247	0.935	0.935

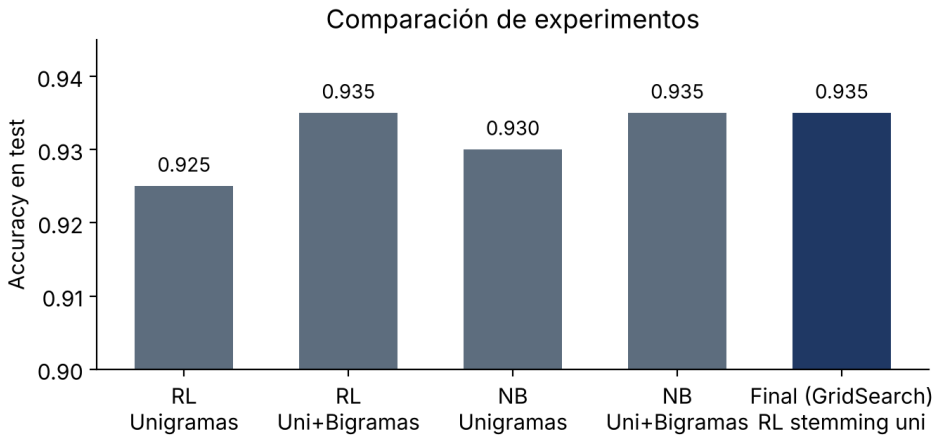


Figura 4. Accuracy de los cuatro experimentos y del modelo final optimizado.

Los bigramas mejoran a la Regresión Logística en un punto (0.925 → 0.935), y Naive Bayes rinde casi igual. Las diferencias son pequeñas: con 200 reseñas de prueba, un punto equivale a solo 2 reseñas. Por eso la elección final no se basó en esta tabla, sino en una búsqueda sistemática con validación cruzada.

4.4. Pipeline y optimización con GridSearchCV

El modelo final es un `Pipeline` de scikit-learn que une las tres etapas en un solo objeto: **limpieza de Keny → TF-IDF → Regresión Logística**. Recibe el texto crudo, lo que evita que la API tenga que repetir la limpieza por su cuenta y garantiza que entrenamiento y producción procesen el texto igual.

`GridSearchCV` probó todas las combinaciones de la siguiente grilla con validación cruzada de 5 particiones sobre las 800 reseñas de entrenamiento: 96 combinaciones × 5 = 480 entrenamientos, optimizando el F1 ponderado.

Tabla 7. Grilla de búsqueda y mejores hiperparámetros.

Hiperparámetro	Valores probados	Mejor valor
Preprocesador	stemming, lematización	**stemming**
Rango de n-gramas	(1,1), (1,2)	**(1,1) unigramas**
min_df (frecuencia mínima)	1, 2	**1**
max_df (frecuencia máxima)	0.9, 1.0	**0.9**
C (inverso de la regularización)	0.1, 1, 10	**10**
Solver	lbfgs, liblinear	**lbfgs**

La mejor configuración obtuvo un **F1 de 0.926 en validación cruzada**. Usa stemming y solo unigramas: como el stemming ya agrupa las variantes, los bigramas añadían columnas sin aportar más información. Un C alto (10) significa poca regularización, es decir, el modelo confía bastante en los pesos que aprende.

4.5. Evaluación final

El modelo optimizado se evaluó una sola vez sobre las 200 reseñas de prueba reservadas:

Tabla 8. Reporte de clasificación en el conjunto de prueba.

Clase	Precisión	Recall	F1	Reseñas
Negativo	0.948	0.920	0.934	100
Positivo	0.922	0.950	0.936	100
Accuracy global			**0.935**	200
F1 ponderado	0.935	0.935	**0.935**	200

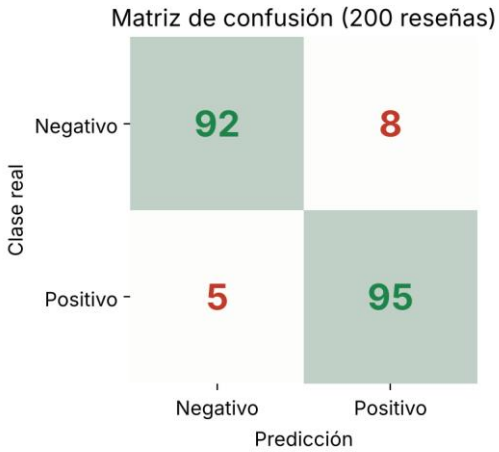


Figura 5. Matriz de confusión: filas = clase real, columnas = predicción.

El modelo acierta **187 de 200 reseñas** (93,5 %). Comete 8 falsos positivos (negativas tomadas como positivas) y 5 falsos negativos. Además, cuando su confianza supera el 80 %

(155 de las 200 reseñas), acierta el **98 %** de las veces: la probabilidad que muestra la interfaz es un buen indicador de qué tan fiable es cada predicción.

4.6. Interpretación del modelo

La Regresión Logística asigna un peso a cada raíz: si es positivo, empuja hacia "Positivo"; si es negativo, hacia "Negativo". La Figura 6 muestra los pesos más fuertes.

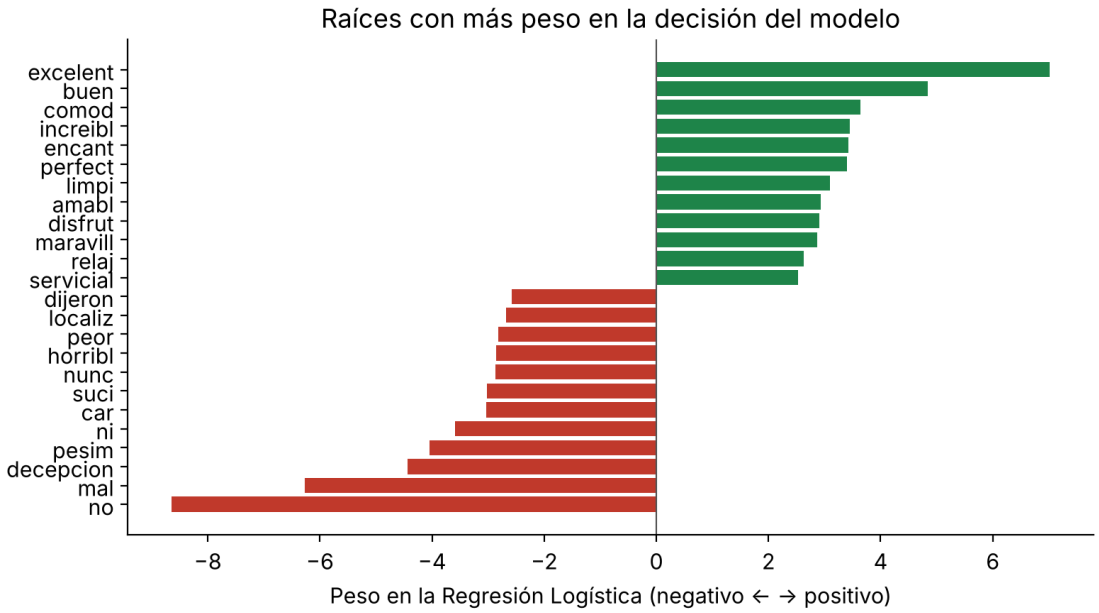


Figura 6. Las 12 raíces con más peso hacia cada clase.

El término con más peso de todo el modelo es **"no"** (−8,6), lo que confirma la decisión de Keny de no eliminar las negaciones. Le siguen "mal", "decepcion" y "pesim". En el lado positivo dominan "excelent", "buen", "comod" y "limpi". El modelo aprendió un vocabulario coherente con el sentido común, sin palabras sospechosas.

4.7. Análisis de errores

Se revisaron a mano las 13 reseñas mal clasificadas. Casi todas pertenecen a uno de tres grupos:

Tabla 9. Tipos de error más frecuentes.

Tipo de error	Ejemplo (resumido)	Real	P(Positivo)
Opinión mixta: elogios y quejas en la misma reseña	"Excelente ubicación, pésimo servicio. El hotel tiene su encanto, pero queda completamente eclipsado…"	Neg.	0.65
Opinión mixta con quejas al final	"Casi excelente. Sin dudarlo es el mejor hotel de Valle de Bravo… pasa una carretera… no tienen aire acondicionado."	Pos.	0.18
Etiqueta dudosa: el texto contradice las estrellas	"Gran fin de semana en pareja. Excelente lugar… instalaciones increíbles con el mejor servicio… Muy recomendable."	Neg.	0.95
Traducción automática o lenguaje poco natural	"…tratarán el timo de cero en usted. Después de regresar el carro un señalando de partida de empleado…"	Pos.	0.06

Las opiniones mixtas son la causa principal: un modelo de bolsa de palabras suma la evidencia de toda la reseña y no sabe que, en "buena ubicación, **pero** pésimo servicio", lo que viene después del "pero" pesa más para el autor. Varios errores tienen probabilidades cercanas a 0,5, es decir, el propio modelo "dudaba". Otros casos no son fallos del modelo, sino de la etiqueta: hay reseñas totalmente elogiosas calificadas con 1–2 estrellas.

4.8. Exportación de artefactos

Al terminar, `train.py` guarda dos archivos que consume el backend:

- `src/models/pipeline_modelo.pkl`: el pipeline completo (limpieza + TF-IDF + Regresión Logística), serializado con `joblib`.
- `src/models/metricas.json`: accuracy, F1, F1 de validación cruzada, matriz de confusión, mejores hiperparámetros y tabla de experimentos.

Todo el entrenamiento usa semilla 42, así que volver a ejecutar `train.py` produce los mismos resultados.

5. Integrante 3 – Backend (API REST con FastAPI)

Responsable: Luis Angel Arze Kao (222007508) · **Carpeta:** `api/` · **Rama:** `feature/backend-api`

5.1. Objetivo

Construir el servidor web que recibe una reseña desde el navegador, la pasa por el modelo entrenado (`pipeline_modelo.pkl`) y devuelve el sentimiento, la probabilidad y el paso a paso del procesamiento NLP para que el frontend lo muestre.

5.2. Estructura y conexión con el equipo

Tabla 10. Archivos del backend.

Archivo	Función
`main.py`	Crea la aplicación FastAPI, carga el modelo con `joblib.load` y define las rutas
`schemas.py`	Modelos Pydantic de entrada y de respuesta, con las validaciones
`config.py`	Configuración por variables de entorno (rutas, orígenes CORS, largo máximo)
`errores.py`	Manejadores de error con mensajes en español y formato único
`registro.py`	Registro (logging) de cada petición con ruta, código y duración
`ejemplos.py`	Reseñas de demostración para la ruta `/ejemplos`
`test_main.py`	Pruebas automáticas con pytest

- El `.pkl` guarda referencias a la clase `NLPPreprocessor` de Keny; por eso la API agrega la raíz del proyecto al `sys.path` antes de cargarlo.
- **Modo simulado:** mientras no existía el modelo, `/predict` respondía siempre "Positivo – 90 %", para que Angela pudiera trabajar desde el primer día. Cuando `pipeline_modelo.pkl` aparece, la API lo usa automáticamente (`modo: "real"`).

5.3. Rutas de la API

Tabla 11. Rutas expuestas por la API (las tres primeras son obligatorias).

Método	Ruta	Descripción
POST	`/predict`	Recibe una reseña, calcula la probabilidad de cada clase y devuelve el veredicto con el paso a paso del NLP
GET	`/metrics`	Devuelve la precisión y demás métricas del modelo (desde `metricas.json`)
GET	`/status`	Indica si el servidor está activo y si el modelo real está cargado
POST	`/predict/lote`	Extra: clasifica hasta 50 reseñas en una petición y cuenta positivas y negativas
GET	`/ejemplos`	Extra: reseñas de demostración (filtro `?categoria=hotel`)

Ejemplo de petición y respuesta (`POST /predict`, resumida, con el modelo real):

```
{ "texto": "El hotel estaba sucio y NO lo recomiendo para nada" }

{
  "sentimiento_predicho": "Negativo",
  "confianza": 0.9866,
  "normalizado": "el hotel estaba sucio y no lo recomiendo para nada",
  "tokens": ["el", "hotel", "estaba", "sucio", "y", "no", "lo", "recomiendo", "para",
"nada"],
  "sin_stopwords": ["hotel", "sucio", "no", "recomiendo", "nada"],
  "lemas": ["hotel", "sucio", "no", "recomiendo", "nada"],
  "raices": ["hotel", "suci", "no", "recom", "nad"],
  "porcentaje": "98.7%",
  "probabilidades": { "Negativo": 0.9866, "Positivo": 0.0134 },
  "modo": "real"
}
```

5.4. Paso a paso del NLP (Requisito 5.2)

La API devuelve los cinco pasos que el frontend debe mostrar, calculados con el mismo `NLPPreprocessor` que usa el modelo. Así el frontend no implementa su propia limpieza.

Tabla 12. Campos de la respuesta que corresponden a cada paso.

Paso	Campo	Contenido
1	`normalizado`	Minúsculas, sin tildes, signos, números ni URLs (con sus etapas intermedias en `pasos.normalizacion_etapas`)
2	`tokens`	El texto separado en palabras
3	`sin_stopwords`	Sin conectores vacíos; las negaciones ("no", "nada") se conservan
4	`raices` / `lemas`	Raíces (stemming, la técnica elegida por GridSearchCV) y lemas
5	`sentimiento_predicho` + `confianza`	Veredicto con su porcentaje de certeza

5.5. Validaciones y manejo de errores

La API rechaza con código **422** las reseñas vacías, las que solo tienen espacios, las que no contienen ninguna letra y las de más de 2000 caracteres. Todos los errores tienen el mismo formato y están en español:

```
{ "error": "Datos inválidos",
  "detalles": [{ "campo": "texto", "mensaje": "La reseña no puede estar vacía." }] }
```

5.6. CORS

El navegador bloquea las llamadas del frontend (puerto 5173) a otro puerto (8000) si la API no envía las cabeceras CORS. Se configuró `CORSMiddleware`, con los orígenes en la variable `ORIGENES_PERMITIDOS` (por defecto `*`).

Se detectó y corrigió un problema: el middleware estaba en una capa interna, así que si la API fallaba con un error 500, la respuesta salía sin cabeceras CORS y el navegador lo mostraba como un error de CORS, ocultando el error real. Ahora CORS es la capa más externa y los errores se devuelven como JSON dentro de ella.

5.7. Funciones adicionales y pruebas

- Configuración por variables de entorno (`api/.env`): puerto, rutas del modelo y de las métricas, orígenes permitidos y largo máximo.
- Registro (logging) de cada petición con su ruta, código de respuesta y duración.
- Documentación automática e interactiva en `/docs` (Swagger), generada por FastAPI.
- Script `api/run.sh` para instalar dependencias, levantar el servidor y correr las pruebas.

Se escribieron **28 pruebas automáticas** con pytest (`api/test_main.py`), todas aprobadas con el modelo real. Cubren la estructura de la respuesta, el modo simulado, el rechazo de textos inválidos, los lotes, las métricas, el estado, los campos planos para el frontend y las cabeceras CORS (también en respuestas de error).

6. Integrante 4 – Frontend (interfaz web)

Responsable: Angela · **Carpeta:** `frontend/`

6.1. Tecnologías y estructura

La interfaz, llamada **Opinómetro**, está hecha con HTML, CSS y JavaScript sin frameworks ni instalación de paquetes: basta con servir la carpeta con `python -m http.server`. El código JavaScript se organizó en módulos con una responsabilidad cada uno:

Tabla 13. Organización del código del frontend.

Archivo	Responsabilidad
`index.html`	Estructura de la página: portada, analizador, laboratorio NLP y métricas
`src/styles.css`	Colores, tipografía, diseño adaptable y animaciones
`src/services/api.js`	Llamadas `fetch()` a la API, con límite de espera de 20 s y mensajes de error claros
`src/main.js`	Coordina el formulario, los ejemplos, las pestañas y el envío
`src/components/result.js`	Tarjeta de resultado (verde/rojo) y probabilidades
`src/components/laboratory.js`	Recorrido paso a paso del NLP y su reproducción
`src/components/text-transform.js`	Animación palabra por palabra de cada etapa
`src/components/batch.js`	Tabla del análisis por lotes
`src/components/metrics.js`	Panel de métricas, matriz de confusión y experimentos

6.2. Pantallas de la aplicación

Las capturas siguientes se tomaron con la aplicación conectada al modelo real.



Figura 7. Portada de Opinómetro.

El usuario escribe la reseña (o carga un ejemplo) y pulsa **Analizar reseña**. La tarjeta de resultado se pinta de **rojo** si la reseña es negativa y de **verde** si es positiva, con el porcentaje de certeza y la barra de probabilidades de ambas clases (Figuras 8 y 9).

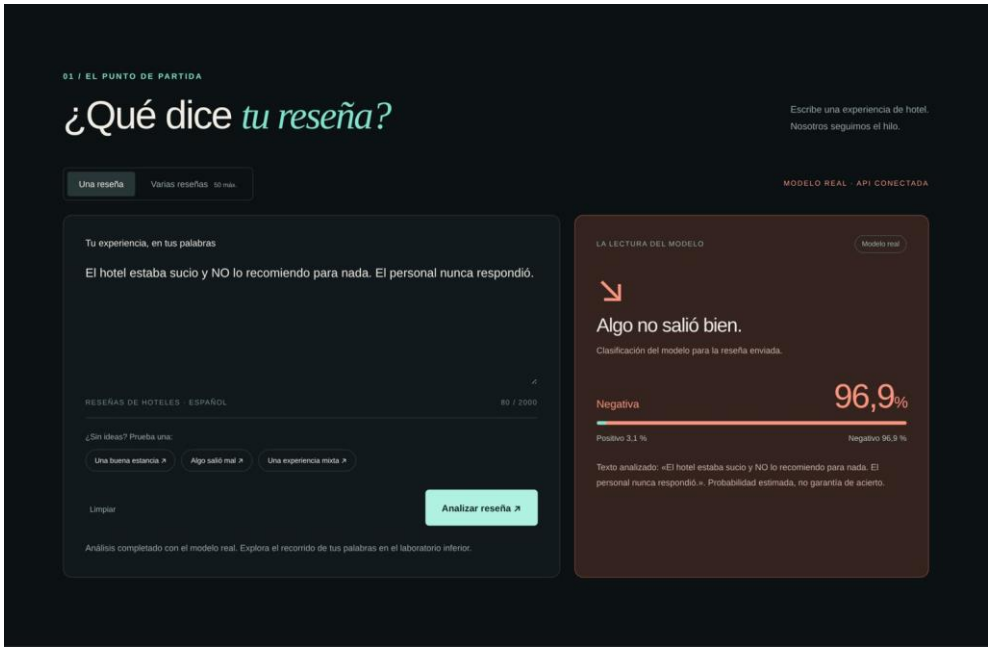


Figura 8. Reseña negativa: tarjeta en rojo con 96,9 % de certeza.

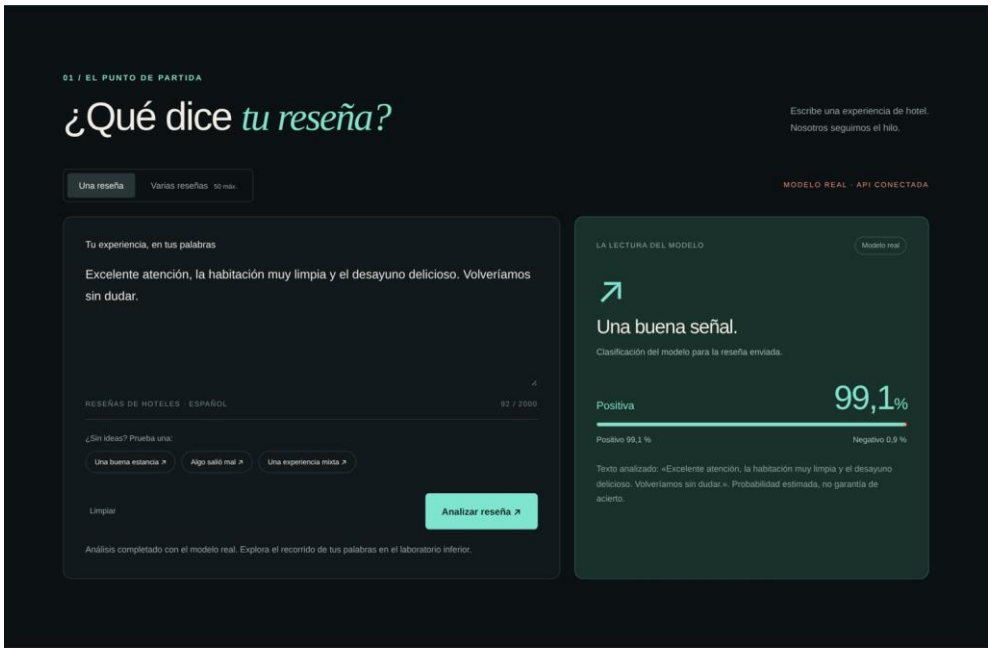


Figura 9. Reseña positiva: tarjeta en verde con 99,1 % de certeza.

La sección **"Un pequeño viaje. Palabra por palabra."** es el laboratorio NLP que pide el profesor. Recorre las cuatro etapas de la limpieza con los datos que devolvió la API, animando cada palabra: las stop words se atenúan y tachan, las negaciones se resaltan y el stemming muestra el par palabra → raíz.



Figura 10. Laboratorio NLP, etapa de tokenización: cada palabra ocupa su casilla.

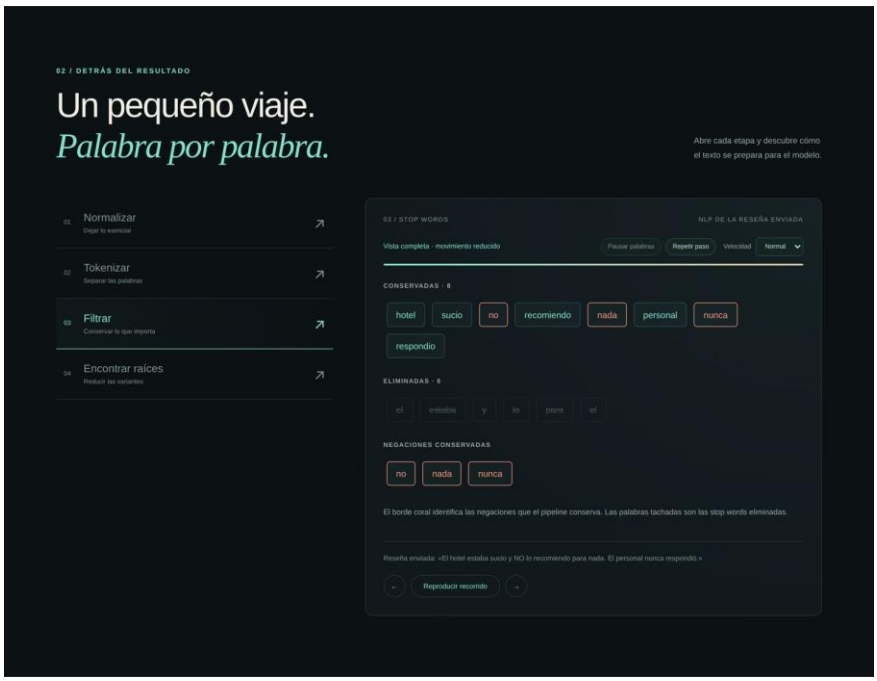


Figura 11. Etapa de filtrado: stop words eliminadas y negaciones ("no", "nada", "nunca") conservadas.

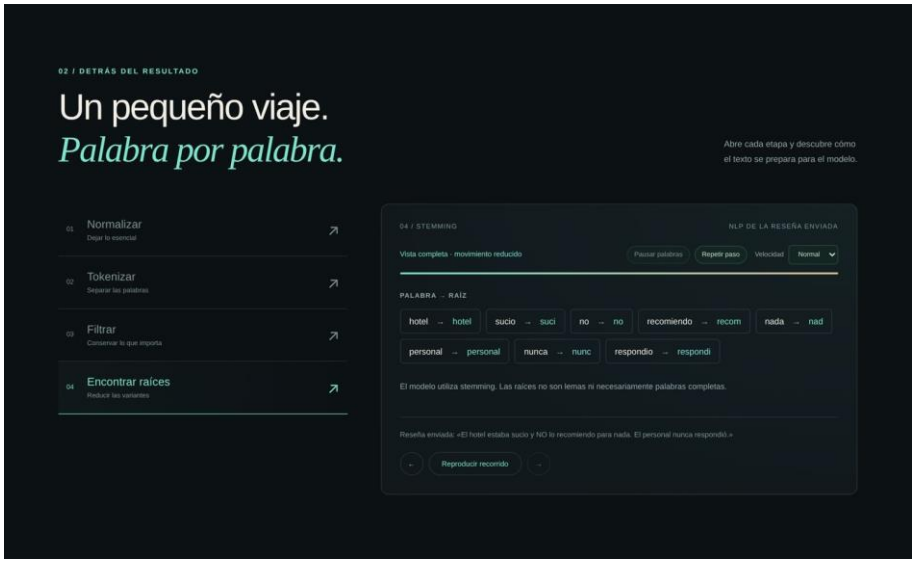


Figura 12. Etapa de stemming: pares palabra → raíz.

La aplicación también permite analizar **varias reseñas a la vez** (una por línea, hasta 50) y muestra un resumen con el conteo de positivas y negativas. Desde la tabla se puede abrir el detalle de cada reseña.

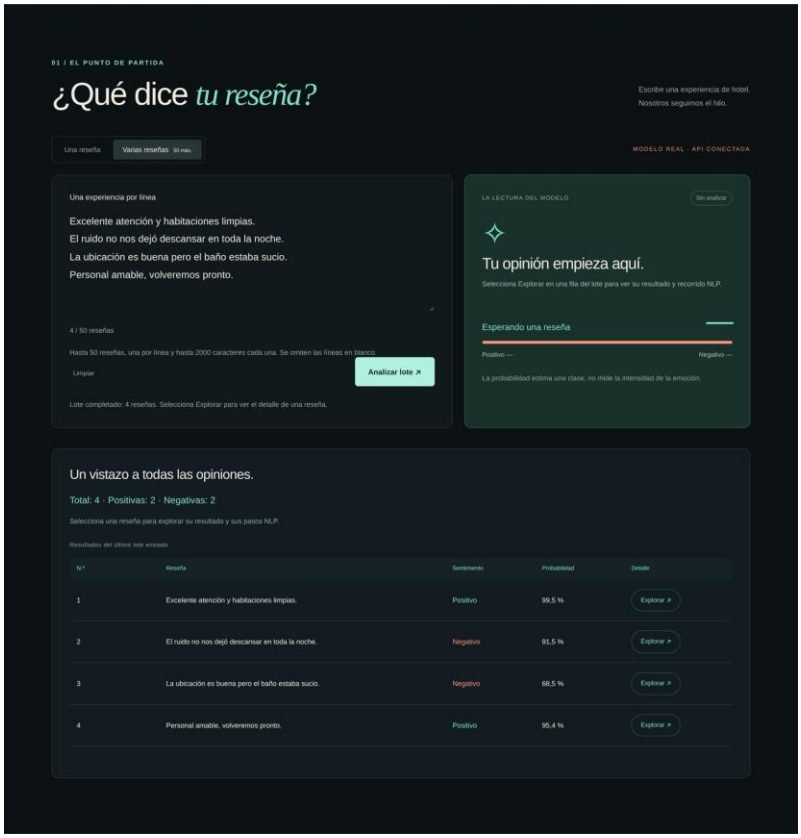


Figura 13. Análisis por lotes de cuatro reseñas.

Por último, la sección **"Menos magia. Más evidencia."** consulta `/metrics` y muestra la exactitud, el F1, la matriz de confusión y la tabla de experimentos, para que el usuario entienda qué tan confiable es el modelo.

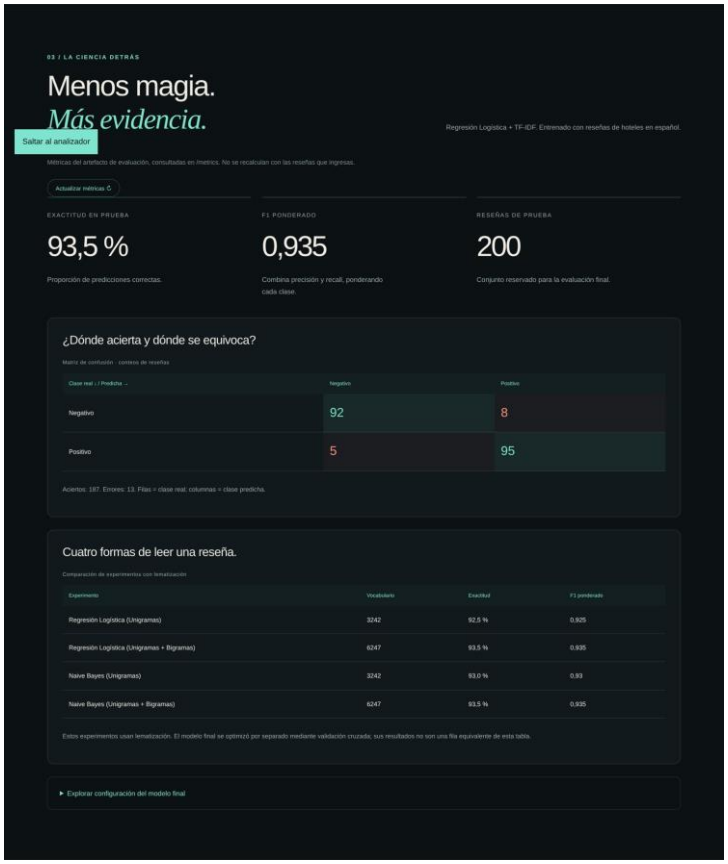


Figura 14. Panel de métricas del modelo.

6.3. Cumplimiento del paso a paso (Requisito 5.2)

Tabla 14. Correspondencia entre los pasos exigidos y la interfaz.

Paso exigido	Cómo lo muestra la interfaz
1. Texto normalizado	Etapa "Normalizar": anima minúsculas, retiro de URLs, tildes, signos y números hasta la frase limpia
2. Tokens	Etapa "Tokenizar": las palabras viajan, en orden, a casillas individuales
3. Stop words eliminadas	Etapa "Filtrar": tacha las stop words y resalta las negaciones conservadas
4. Stemming	Etapa "Encontrar raíces": muestra cada palabra junto a su raíz
5. Resultado y probabilidad	Tarjeta verde (positivo) o roja (negativo) con el porcentaje de certeza

6.4. Validaciones, accesibilidad y seguridad

- No permite enviar reseñas vacías ni de más de 2000 caracteres, y muestra los mensajes de error de la API.
- Distingue el modo real del simulado y avisa si la API está desconectada, con opción de reintentar.
- Si el usuario edita la reseña después de analizarla, advierte que el resultado corresponde al texto anterior.
- Los textos se insertan con `textContent`, nunca como HTML, para evitar inyección de código.

- Navegación por teclado, atajo Ctrl+Enter, diseño adaptable a celulares y opción para pausar las animaciones (respeta la preferencia de "movimiento reducido" del sistema).

7. Integración y verificación

La integración se hizo uniendo las ramas de los cuatro integrantes. Para comprobar que las piezas encajan, se verificó lo siguiente:

- **Modelo reproducible:** al cargar `pipeline_modelo.pkl` y evaluarlo sobre la misma partición (semilla 42), se obtiene exactamente accuracy 0.935 y la matriz [[92, 8], [5, 95]] registradas en `metricas.json`.
- **Limpieza coherente:** en las 200 reseñas de prueba, las raíces que la API muestra al usuario coinciden con las que usa internamente el modelo (cero diferencias). Lo que se ve en pantalla es exactamente lo que procesa el clasificador.
- **Backend:** 28 pruebas automáticas aprobadas con el modelo real.
- **Extremo a extremo:** desde la interfaz se probaron reseñas positivas, negativas, lotes, textos inválidos, API desconectada y API sin modelo (modo simulado).

Para ejecutar la aplicación se necesitan dos terminales, una para cada servidor:

```
pip install -r api/requirements.txt
python -c "import nltk; nltk.download('stopwords')"

# Terminal 1: backend  ->  http://127.0.0.1:8000/docs
uvicorn api.main:app --host 127.0.0.1 --port 8000

# Terminal 2: frontend ->  http://127.0.0.1:5173
python -m http.server 5173 --bind 127.0.0.1 --directory frontend

# Pruebas del backend
python -m pytest api/test_main.py -v
```

8. Resultados y discusión

Tabla 15. Resumen de resultados.

Indicador	Resultado
Reseñas utilizadas	1000 reales (500 positivas y 500 negativas), Rest-Mex 2025
Modelo final	Stemming + TF-IDF (unigramas) + Regresión Logística (C = 10)
F1 ponderado en validación cruzada	0.926
Accuracy en prueba	**0.935** (187 de 200 aciertos)
F1 ponderado en prueba	**0.935**
Rutas de la API	3 obligatorias + 2 adicionales; 28 pruebas aprobadas
Paso a paso del NLP	5 pasos mostrados en la interfaz, calculados por el backend

Un modelo clásico y simple (TF-IDF con Regresión Logística) alcanza un 93,5 % de exactitud sobre reseñas reales, con un comportamiento equilibrado entre clases (F1 de 0.934 y 0.936). Las decisiones de preprocesamiento tuvieron un efecto medible: conservar las negaciones le dio al modelo su variable más importante ("no"), y el stemming redujo el vocabulario a casi la

mitad sin perder exactitud. Al ser un modelo lineal, se puede explicar con sus pesos, lo que encaja con el objetivo de transparencia del proyecto.

9. Limitaciones y trabajo futuro

- El modelo se entrenó solo con reseñas de **hoteles** en español de México. Con reseñas de restaurantes o productos, o con otras variantes del español, el rendimiento puede bajar.
- Es **binario**: no tiene clase neutral ni mide la intensidad del sentimiento. Las opiniones mixtas son su principal fuente de error.
- La bolsa de palabras ignora el orden y no maneja bien el sarcasmo ni los contrastes ("bonito, pero…"). Modelos basados en transformers en español (como BETO) podrían capturar mejor el contexto.
- La lematización se basa en un diccionario pequeño; un lematizador completo (por ejemplo, spaCy) permitiría una comparación más justa con el stemming.
- Con 200 reseñas de prueba, cada punto porcentual equivale a 2 reseñas; un conjunto mayor daría estimaciones más precisas.
- La API no explica cada predicción individual (qué palabras pesaron más); se podría añadir mostrando los pesos de la Figura 6 para cada reseña.

10. Conclusiones

Se cumplió el objetivo de construir una aplicación web completa que clasifica el sentimiento de reseñas de hoteles en español y que muestra de forma visual cómo se transforma el texto en cada etapa. El pipeline reproducible, que va desde la limpieza hasta el modelo exportado, obtuvo un 93,5 % de exactitud en datos que no vio durante el entrenamiento.

La división en cuatro roles con un contrato claro entre etapas (funciones de limpieza → `.pkl` → JSON de la API → interfaz) permitió trabajar en paralelo. El modo simulado del backend y el pipeline que encapsula la limpieza fueron las decisiones que más facilitaron la integración: el frontend pudo avanzar desde el primer día, y lo que el usuario ve en pantalla coincide exactamente con lo que procesa el modelo.

11. Declaración de uso de inteligencia artificial

De acuerdo con la consigna, se declara el uso de herramientas de IA generativa durante el proyecto:

- El frontend y un ajuste en la descarga de stop words de NLTK se desarrollaron con asistencia de Codex (OpenAI). El equipo revisó el código generado.
- La redacción de este informe, las figuras de análisis y las capturas se prepararon con apoyo de Claude (Anthropic), a partir del código, los datos y la documentación del repositorio. Las cifras se recalcularon a partir del modelo entregado.

Los datos, el diseño del pipeline, las decisiones de preprocesamiento y los resultados reportados corresponden al trabajo del equipo y pueden reproducirse con los scripts del repositorio.

12. Contribuciones e historial

Tabla 16. Aportes de cada integrante.

Integrante	Aportes principales (según el historial de Git)
Keny	Estructura de carpetas en cascada; `preprocess.py` (limpieza, stop words con negaciones, stemming y lematización); notebook de EDA; README e informe técnico inicial
Nicolás	`train.py`: carga y división estratificada, experimentos de n-gramas y modelos, pipeline con GridSearchCV, exportación del `.pkl` y `metricas.json`; reemplazo del dataset sintético por reseñas reales (Rest-Mex 2025); `requirements.txt`
Luis Angel	API FastAPI completa: rutas `/predict`, `/metrics`, `/status`, `/predict/lote` y `/ejemplos`; configuración, esquemas Pydantic, errores, registro, CORS, 28 pruebas, `run.sh` y documentación; integración de las ramas
Angela	Interfaz Opinómetro: analizador individual y por lotes, laboratorio NLP animado, panel de métricas, validaciones, accesibilidad y guía de demostración

Historial de commits del backend (rama `feature/backend-api`):

1. Servidor FastAPI con rutas `/predict`, `/metrics` y `/status`
2. Configuración con variables de entorno
3. Modelos de respuesta con Pydantic
4. Manejo de errores personalizado
5. Registro de peticiones
6. Ruta `POST /predict/lote`
7. Ruta `GET /ejemplos`
8. Pruebas con pytest
9. Script `run.sh`
10. Documentación de la API
11. JSON plano con el proceso NLP (lemas, raíces) y CORS también en errores

Referencias

- Rest-Mex 2025: Sentiment Analysis of Mexican Tourist Texts. Conjunto de datos en Hugging Face (vg055/Rest-Mex2025), licencia CC BY 4.0.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Bird, S., Klein, E. y Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. (NLTK)
- Porter, M. F. (2001). Snowball: A language for stemming algorithms. snowballstem.org
- FastAPI. Documentación oficial. fastapi.tiangolo.com
- Manning, C. D., Raghavan, P. y Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. (TF-IDF)